

ON SCALING LINEAR PROGRAMMING PROBLEMS*

J.A. TOMLIN

Stanford University, Stanford, Calif., U.S.A.

Received 5 April 1974

Revised manuscript received 15 January 1974

The scaling of linear programming problems remains a rather poorly understood subject (as indeed it does for linear equations). Although many scaling techniques have been proposed, the rationale behind them is not always evident and very few numerical results are available. This paper considers a number of these techniques and gives numerical results for several real problems. Particular attention is given to two "optimal" scaling methods, giving results on their speed and effectiveness (in terms of their optimality criteria) as well as their influence on the numerical behavior of the problem.

1. Introduction

Although scaling is often (though by no means always) applied to linear programming (LP) problems, very little is really understood about it. Certain scaling methods are reputed to be more effective than others but comparative figures, or even the criteria by which the effectiveness is judged, seem to appear nowhere in the open literature. Similarly some problems are supposed to behave disastrously when unscaled and quite well when scaled and, more frequently, vice-versa. Again very little documented evidence is available.

The aim of this paper is to present some data on, and interpretation of, the effect of scaling methods on a range of real LP problems. We do not claim to have solved any of the fundamental problems concerned with scaling. With the present state of knowledge it seems to be possible only to present experimental evidence and some recommendations.

* Research and reproduction of this report was partially supported by the U.S. Atomic Energy Commission Contract AT(04-3)-326 PA # 18; and National Science Foundation, Grant GJ 30408X1.

The constraints of a linear program will be represented as

$$[I, A] \mathbf{x} = \mathbf{b}, \quad (1.1)$$

$$L_j \leq x_j \leq U_j \quad (j = 1, \dots, n), \quad (1.2)$$

where A is $m \times (n - m)$ and $n > m$. For those variables with finite L_j it is customary to normalize the lower bounds to zero with modifications to their upper bounds and the right-hand side $\mathbf{b}$.

When we speak of scaling an LP, we mean a transformation of the linear constraints (1.1) by two diagonal matrices

$$R = \text{diag}(r_1, \dots, r_m), \quad C = \text{diag}(c_1, \dots, c_n)$$

to the form

$$R[I, A] C\bar{\mathbf{x}} = R\mathbf{b} \quad (1.3)$$

which carries with it a transformation of the variables

$$\bar{\mathbf{x}} = C^{-1}\mathbf{x} \quad (1.4)$$

and of the finite adjusted bounds

$$0 \leq \bar{x}_j \leq \frac{U_j - L_j}{c_j}. \quad (1.5)$$

It is important to note that to preserve the m logical (unit) columns we require that

$$c_i = r_i^{-1} \quad (i = 1, \dots, m). \quad (1.6)$$

That is, the column scales of the logicals are determined by the row scales of the structural matrix A .

2. Reasons for scaling

Three reasons are commonly advanced for scaling linear programs:

(a) Compact representation of the bounds on variables. (From (1.5) we see that the column scales may be chosen so that all finitely bounded variables have a common upper bound, say u . Often u is arbitrarily chosen to be unity (see [11]).)

(b) To reduce the number of simplex iterations required to solve the LP.

(c) To improve the numerical behavior of the simplex algorithm.

The first of these can be dealt with quite quickly. If there is one result which can be stated with any firmness in this area it is that the common upper bound technique is short-sighted and inadvisable. In return for a relatively trivial saving in main memory and programming convenience very serious distortions of the data may be introduced, particularly if some $(U_j - L_j)$ are very large or very small; leading to gratuitous opportunities for numerical troubles. Hentges [9] has given details of a problem with 721 rows and 1156 structural columns which behaves particularly badly using this form of scaling even though only 30 of the variables have upper bounds. By intelligent choice of u and of the r_i and c_j not dictated by bound considerations he was eventually able to successfully solve the problem. This does not alter the conclusion that the technique is undesirable and we shall not consider it further.

Much more to the point is reason (b). Most LP systems choose the column to enter the basis at any simplex iterations from some subset of those with the most negative reduced costs:

$$d_j = \begin{cases} \pi_j & \text{for } j \leq m, \\ \pi' a_j & \text{for } j > m, \end{cases} \quad (2.1)$$

where $\pi' = c'B^{-1}$ and B is the current basis and c' some vector (see Beale [2]). Clearly if the matrix is badly scaled and the structural columns a_j are of vastly different magnitude, their d_j values will give little indication of the potential contribution a column can make in improving the objective function. Similarly poor row scaling (from (1.6)) will give misleading significance to the d_j values for the logical columns. The net result may be that activities of marginal profitability are constantly introduced and subsequently dropped from the basis due to their deceptively large and negative d_j values while extremely profitable activities may have deceptively small d_j 's and be introduced very late. Despite the plausibility of this argument fixed pre-scaling of the type discussed here does not always seem to have as great an effect as one might expect. We shall return to this point in a later section.

The aim of improving the numerical behavior of the simplex algorithm is by far the most important (and ambitious) of the reasons for scaling and will be the principle concern of the remainder of this paper. The breakdown of the simplex algorithm in the face of rounding error, though extremely rare, normally manifests itself in loss of feasibility in the solution or singularity of the basis. The first line of defense of a production LP system against these disasters is a set of upward of a dozen tolerances. Some of these

tolerances (particularly those concerned with inversion) may be relative, but most are absolute. For example, in choosing the pivot row for an updated column

$$\alpha = B^{-1}a_q \quad (2.2)$$

chosen to enter the basis, an element will only be considered as a pivot if $|\alpha_i| > \varepsilon_p$ for some small $\varepsilon_p > 0$. In forming a transformation from α we then consider any $|\alpha_i| < \varepsilon_0$ to be zero. In assigning values to these tolerances some assumptions must be made about the distribution of the matrix elements. Usually they are assumed to be fairly closely distributed about unity; an assumption which can easily be rendered false by poor scaling. We can also see that since the d_j 's are asserted to be negative only if they are less than some tolerance $-\varepsilon_d$ it is conceivable for poor scaling to result in a premature declaration of optimality.

Apart from the relatively straightforward matter of tolerances there are other questions relating to the accuracy of the solutions of linear systems which are particularly relevant to LP inversion routines [16] and it is to these we now turn.

3. Scaling linear equations

If the process of scaling linear programs is poorly understood, the situation with ordinary linear equations cannot be said to be a great deal better. Some results are however available.

Let us consider the $m \times m$ matrix B and the set of equations

$$Bx = b. \quad (3.1)$$

In solving such a system by Gaussian elimination, Wilkinson [17] has shown that the solution obtained is that of a perturbed system

$$(B + \delta B)(x + \delta x) = b. \quad (3.2)$$

The relative error in the computed solution $(x + \delta x)$ is then bounded by

$$\frac{\|\delta x\|}{\|x + \delta x\|} \leq \kappa(B) \frac{\|\delta B\|}{\|B\|}, \quad (3.3)$$

where

$$\kappa(B) = \|B\| \|B^{-1}\| \quad (3.4)$$

is the condition number of B and we use throughout the l_∞ norm:

$$\|x\| = \max_i |x_i|, \quad \|B\| = \max_i \left\{ \sum_j |b_{ij}| \right\},$$

which is by far the cheapest to compute in practice.

Slightly extending Wilkinson's results, Reid [12] has shown that whatever the pivot strategy used $\|\delta B\|$ is bounded by a constant proportional to the largest number created in the course of the elimination, which we denote by g . As a first order approximation then, from (3.3), we have that the relative error in the true solution x is bounded by a constant proportional to

$$\frac{g \kappa(B)}{\|B\|}.$$

Following this observation one scaling strategy is to try and minimize the condition number, assuming that g can be kept small by employing partial or complete pivoting. This is the approach taken by Van der Sluis [15], who shows that $\kappa(B)$ can be minimized by scaling so that each row of B has equal l_1 norm [14]. This procedure does not seem very practical for LP, where the basis changes at each iteration in a quite unpredictable way.

Another firm result in the scaling of linear equations, due to Bauer, states that the actual error in the solution of a set of linear equations by Gaussian elimination with a fixed pivot sequence is unchanged if the matrix is scaled by integer powers of the floating point base of the machine in use [1]. This implies that the accuracy of the solution is related only to the sequence of pivot choices. A good scaling strategy is therefore one which admits a good choice of pivots, preventing g from becoming unreasonably large. The usual criterion for choosing a pivot in Gaussian elimination is that it should be within some relative pivot tolerance (say u) of the largest available pivot element in the column (see e.g., [16]). The presumption is that this strategy will be effective in keeping g small only if the elements of the original matrix B are comparable in size, and that scaling should be aimed at attaining this goal.

4. "Well scaled" matrices

It is easy to give a definition of badly scaled matrices—the elements are of vastly different magnitude. On the other hand there appears to be no obvious definition of a well scaled matrix. We shall adopt the rather "fuzzy"

definition that a matrix is well scaled if the elements are as nearly of the same magnitude as possible. This "definition" is compatible with the argument at the end of the previous section as well as asserting the opposite of our definition of bad scaling. This notion of good scaling also has the advantage that it can be reasonably applied to the entire LP coefficient matrix A with the expectation that the property will largely extend to the individual bases B . There will also be a tendency for the bases to have rows with comparable l_1 norms, and hence for their condition numbers to be reduced.

Unfortunately, although the concept of equating good scaling with making the elements of as nearly the same magnitude as possible is quite widely accepted, there is no complete agreement as to what this means. Fulkerson and Wolfe [6] and Orchard-Hays [11] assert that the matrix is well scaled if, considering only the nonzero a_{ij} ,

$$\max_{ij} |a_{ij}| / \min_{ij} |a_{ij}| \leq R \quad (4.1)$$

for some acceptable R . Hamming [7], and Curtis and Reid [4] assert a matrix to be well scaled if the elements have

$$\sum_{a_{ij} \neq 0} (\log |a_{ij}|)^2 \leq V \quad (4.2)$$

for some acceptable V . Curtis and Reid in particular argue that the second criterion is preferable on the grounds that a few large or small elements may be acceptable if the remaining elements are of comparable size.

In any scaling procedure it is advisable to avoid tampering with the significance of the user's data and hence to use only scaling factors which are integer powers of the floating point base. Assuming binary arithmetic our original data will then be in the form

$$a_{ij} = 2^{e_{ij}} f_{ij} \quad (\frac{1}{2} \leq f_{ij} < 1) \quad (4.3)$$

and our scaling factors of the form

$$r_i = 2^{u_i}, \quad c_j = 2^{v_j}, \quad (4.4)$$

where e_{ij} , u_i and v_j are integers. The scaled matrix will then have exponents

$$e'_{ij} = u_i + v_j + e_{ij}. \quad (4.5)$$

In measuring the effectiveness of scaling algorithms we shall therefore be interested in the following quantities

$$e_{\max} = \max e'_{ij}, \quad (4.6)$$

$$e_{\min} = \min e'_{ij}, \quad (4.7)$$

$$\mu = \text{mean}(e'_{ij}), \quad (4.8)$$

$$\sigma^2 = \text{variance}(e'_{ij}). \quad (4.9)$$

When referring to the unscaled matrices, e'_{ij} is of course replaced by the original e_{ij} in the above expressions.

Using this form of scaling we can (with a very slight degree of approximation) replace our measures R and V of good scaling by

$$d = e_{\max} - e_{\min} \quad (4.10)$$

and σ^2 (taking logarithms to base 2 in (4.2)). One method of measuring the effectiveness of a scaling algorithm is then the extent to which it reduces one or both of the quantities d and σ^2 .

5. Empirical methods

Several scaling methods have been used or advocated for some years, mainly on empirical grounds unsupported by computational results in the open literature. Foremost among these methods have been the following:

Equilibration

Each row is scaled to make the largest element of order unity, followed by a similar scaling of the columns.

Geometric mean

For each row we compute $(\max_j |a_{ij}| \min_j |a_{ij}|)^{1/2}$ and divide the row by this number or the nearest power of 2. This is followed by a similar column scaling.

Arithmetic mean

Each row is divided by the arithmetic mean of the elements in that row, or by the nearest power of 2. Again this is followed by a similar treatment of the columns.

Either of the latter techniques may be followed by equilibration.

Slate [13] has carried out a set of experiments with these methods and kindly provided the author with her results. Although these experiments were not carried out using exponent modification as in (4.3)–(4.5) and the results are tabulated in the form of number of elements falling in a certain range, they can be and have been converted to a very close approximation of the form used in this article.

The statistics of the LP matrices used in this set of experiments are given in Table 4.1. The effects on the LP matrices of the different scaling

Table 5.1
Problem statistics—First sample

Problem	S1	S2	S3	S4	S5	S6	S7	S8
Rows	204	126	342	928	822	767	2466	5000
Structural columns	245	248	893	1456	2139	1079	6114	~ 10000
Non-zero elements	1763	2081	4729	11639	13738	14843	45026	136371

methods used are presented in Tables 4.2–4.9. The scaling methods used are denoted in these tables as follows:

- : Unscaled,
- E: Equilibrated,
- GM: Geometric Mean,
- GM&E: Geometric Mean followed by Equilibration,
- AM: Arithmetic Mean,
- AM&E: Arithmetic Mean followed by Equilibration.

Table 5.2
Results for S1

	μ	σ^2	$e_{\max}$	$e_{\min}$	d
Scaling:					
—	1.17	12.13	8	–9	17
E	–1.30	4.89	1	–9	10
GM	0.44	3.29	5	–3	8
GM&E	–1.00	2.15	1	–5	6
AM	0.28	3.77	2	–3	5
AM&E	–1.06	1.99	1	–7	8

Table 5.3
Results for S2

	μ	σ^2	$e_{\max}$	$e_{\min}$	d
Scaling:					
—	-1.16	14.02	8	-10	18
E	-2.55	8.09	1	-17	18
GM	0.66	3.70	5	-4	9
GM&E	-1.54	3.51	1	-8	9
AM	-0.07	4.43	3	-9	12
AM&E	-1.64	3.76	1	-11	12

Table 5.4
Results for S3

	μ	σ^2	$e_{\max}$	$e_{\min}$	d
Scaling:					
—	-1.03	10.09	7	-9	16
E	-1.78	9.39	1	-15	16
GM	0.43	3.78	7	-6	13
GM&E	-1.22	3.52	1	-12	13
AM	0.11	4.44	3	-12	15
AM&E	-1.12	3.25	1	-14	15

Table 5.5
Results for S4

	μ	σ^2	$e_{\max}$	$e_{\min}$	d
Scaling:					
—	-1.62	10.03	7	-12	19
E	-2.31	8.65	1	-13	14
GM	0.44	4.51	6	-5	11
GM&E	-1.61	4.09	1	-9	10
AM	-0.03	5.16	4	-11	15
AM&E	-1.52	3.80	1	-12	13

The general trend of these results is that both GM and AM scaling perform better than Equilibration in reducing both d and σ^2 , with GM performing better than AM overall on both counts. This substantiates the view (see e.g., Orchard-Hays [11]) that GM scaling is superior to the traditional equilibration, at least by the criteria used here. It is also apparent from these results that it is generally beneficial to follow both GM and AM scaling by equilibration, though the additional reductions in d and σ^2 are

not nearly as significant as those produced by the first pass. It is noticeable, however, that when equilibration is carried out, the mean exponent value μ can fall significantly below zero (see particularly problem S6). If the absolute tolerances of the LP system are set assuming elements with a mean of unity as discussed earlier this could possibly lead to trouble.

In the light of these results it seems reasonable to recommend that if one of these simple (and inexpensive) scaling methods is to be used, the

Table 5.6
Results for S5

	μ	σ^2	$e_{\max}$	$e_{\min}$	d
Scaling:					
—	0.45	9.52	8	-12	20
E	-1.93	8.09	1	-16	17
GM	0.61	3.25	7	-7	14
GM&E	-1.35	3.17	1	-12	13
AM	0.13	4.07	4	-12	16
AM&E	-1.30	3.00	1	-15	16

Table 5.7
Results for S6

	μ	σ^2	$e_{\max}$	$e_{\min}$	d
Scaling:					
—	-2.47	20.65	11	-18	29
E	-6.03	22.43	1	-22	23
GM	0.19	11.21	11	-10	21
GM&E	-4.37	12.28	1	-17	18
AM	-1.62	11.51	5	-17	22
AM&E	-4.36	13.15	1	-19	20

Table 5.8
Results for S7

	μ	σ^2	$e_{\max}$	$e_{\min}$	d
Scaling:					
—	0.73	3.46	7	-16	23
E	-0.27	5.26	1	-22	23
GM	0.43	3.70	7	-6	13
GM&E	-1.53	5.16	1	-12	13
AM	0.51	4.61	2	-17	19
AM&E	-0.93	2.78	1	-18	19

Table 5.9
Results for S8

	μ	σ^2	$e_{\max}$	$e_{\min}$	d
Scaling:					
—	1.29	17.53	15	-12	27
E	-2.45	7.91	1	-13	14
GM	0.82	4.45	6	-5	11
GM&E	-1.59	3.45	1	-9	10
AM	-0.16	5.32	5	-11	16
AM&E	-1.67	3.73	1	-12	13

GM method, optionally followed by equilibration, is the most satisfactory with the provision that all the elements have their exponent modified by a constant to give a mean absolute value close to unity.

6. De Buchet's method

A method part way between those of the previous section, which are not specifically designed to minimize either d or σ^2 , and the "optimal" methods to be discussed in the following sections is the method due to De Buchet [3]. With this method the scaling factors r_i , c_j are chosen to

$$\min_{r_i, c_j} \sum_{a_{ij} \neq 0} \left\{ r_i a_{ij} c_j + \frac{1}{r_i a_{ij} c_j} \right\}^2. \quad (6.1)$$

The effect of such a choice of r_i , c_j is clearly to cluster the scaled matrix elements about unity and hence to reduce σ^2 . De Buchet uses an (unspecified) iterative method to solve the normal equations for (6.1), which apparently converges very quickly, and gives results for a problem with 1654 rows, 6096 columns and 44107 non-zero elements. From the graphical data presented in [3], σ^2 was reduced from 13.83 to 3.9 and d from 28 to 16, and the method appears extremely effective.

7. "Optimal" scaling methods

Given that small values of d and/or σ^2 are indeed good measures of how well scaled a matrix is, it is logical to try methods which minimize one or the other of these quantities. Although we cannot of course minimize

both simultaneously it is reasonable to expect that the minimization of one will tend to reduce the other. Algorithms have in fact been devised by Fulkerson and Wolfe [6] for minimizing d and Curtis and Reid [4] for minimizing σ^2 .

The Fulkerson–Wolfe method

We wish to find integer u_i, v_j which give

$$\begin{aligned}\min d &= \min \left\{ \max_{i,j} e'_{ij} - \min_{i,j} e'_{ij} \right\} \\ &= \min \left\{ \max_{i,j} (u_i + v_j + e_{ij}) - \min_{i,j} (u_i + v_j + e_{ij}) \right\}.\end{aligned}$$

This is equivalent to solving

$$\min_{u_i, v_j} \max_{i,j} (u_i + v_j + e_{ij})$$

subject to

$$u_i + v_j + e_{ij} \geq 0 \quad (i = 1, \dots, m; j = 1, \dots, n)$$

which in turn is equivalent to solving

$$\begin{aligned}\text{minimize} \quad & w, \\ \text{subject to} \quad & 0 \leq u_i + v_j + e_{ij} \leq w, \\ & u_i, v_j \text{ integer} \quad (i = 1, \dots, m; j = 1, \dots, n).\end{aligned}$$

This problem may be solved by a labelling algorithm reminiscent of those used in network flow problems. Without going into complete detail (see [6]) the algorithm proceeds by seeking a closed path of elements in the matrix whose row and column scales may be modified to reduce w . In a typical out-of-core LP system this involves many scans or passes of the matrix and hence a great deal of I/O (input/output).

The minimum w is the minimum d we seek and the optimal values u_i, v_j may be modified by a constant to bring the mean exponent close to zero.

The Curtis–Reid method

The minimization of σ^2 is equivalent to finding

$$\min_{u_i, v_j} \sum_{a_{ij} \neq 0} (u_i + v_j + e_{ij})^2. \quad (7.1)$$

To solve (7.1) we solve the normal equations:

$$\sum_{j \mid a_{ij} \neq 0} (u_i + v_j) = - \sum_{j \mid a_{ij} \neq 0} e_{ij} \quad (i = 1, \dots, m), \quad (7.2)$$

$$\sum_{i \mid a_{ij} \neq 0} (u_i + v_j) = - \sum_{i \mid a_{ij} \neq 0} e_{ij} \quad (j = 1, \dots, n). \quad (7.3)$$

Putting this system in matrix form we have

$$\begin{bmatrix} M & Z \\ Z^T & N \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \end{bmatrix} = \begin{bmatrix} \mathbf{s} \\ \mathbf{t} \end{bmatrix}, \quad (7.4)$$

where Z is a matrix with

$$z_{ij} = \begin{cases} 1 & \text{if } a_{ij} \neq 0, \\ 0 & \text{otherwise.} \end{cases} \quad (7.5)$$

M and N are diagonal matrices whose diagonal elements m_i, n_j are the number of non-zeros in each row and column of A respectively, and $\mathbf{s}$ and $\mathbf{t}$ are vectors corresponding to the right-hand sides in (7.2), (7.3).

The system (7.4) must now be solved to obtain values of u_i, v_j which are then rounded to integer values. A mean exponent close to zero will clearly also result.

At first sight it appears ridiculous to solve an $(m + n) \times (m + n)$ system of equations merely to obtain scaling factors for an $m \times n$ linear program. However, using a special version of the conjugate gradient algorithm designed to take advantage of the structure of the matrix in (7.4), Curtis and Reid were able to obtain solutions of acceptable accuracy in only 8 to 10 iterations for matrices with $(m + n)$ between 45 and 199. Applying this technique in an LP system only one scan or pass of the LP problem matrix is required per iteration of the conjugate gradient method. Furthermore the number of iterations required, even using a stricter convergence criterion than that given in [4], seems to remain in the range 8–10 whatever the dimensions of the LP matrix, as will be seen in Table 7.2.

To determine the effectiveness of these two methods in reducing d and σ^2 both were experimentally implemented in Scientific Control Systems' UMPIRE Mathematical Programming System for the Univac 1108. The set of problems used in these experiments are described in Table 7.1. These problems are the same set as those used in an earlier set of experiments on inversion [16] with one slight modification in that some simple upper bounds were detected and removed from the matrix of problem A.

The results of these experiments are presented in Table 7.2. It is clear

Table 7.1
Problem statistics—second sample

Problem	A	B	C	D	E	F
Rows	587	822	1016	1315	2188	3070
Structural columns	1350	1571	3080	3174	6333	9157
Non-zero elements	9926	11127	15424	12190	47505	74814

that, contrary to first expectations, the Curtis–Reid algorithm (denoted C–R) is far faster than the Fulkerson–Wolfe algorithm (denoted F–W), even with the enhancement for F–W suggested in [4] incorporated, as it is here. Furthermore, the number of matrix passes (and hence amount of I/O) required by F–W is very erratic while the number of passes for C–R remains almost unbelievably constant for $(m + n)$ ranging between nearly 2 000

Table 7.2
Performance of Fulkerson–Wolfe and Curtis–Reid algorithms

Problem	Scaling	μ	σ^2	d	Matrix passes	Time (in seconds)
A	—	−0.14	11.51	24	—	—
	F–W	−0.23	5.52	8	302	146
	C–R	−0.07	1.05	11	10	23
B	—	0.55	9.10	20	—	—
	F–W	−0.33	4.16	8	249	273
	C–R	0.02	1.16	11	10	14
C	—	−1.14	11.55	17	—	—
	F–W	−0.27	4.16	7	696	679
	C–R	−0.08	0.71	13	8	20
D	—	1.01	5.18	23	—	—
	F–W	−0.04	5.45	9	206	153
	C–R	0.04	0.86	14	10	39
E	—	0.82	13.62	26	—	—
	F–W	0.00	6.48	11	195	433
	C–R	0.03	1.00	20	10	96
F	—	0.87	12.30	21	—	—
	F–W	−0.82	7.80	12	334	1227
	C–R	0.01	1.02	21	10	172

and over 12 000. It thus appears that the Fulkerson and Wolfe algorithm is too expensive for practical use, while the Curtis and Reid algorithm is quite economical, requiring only a tiny fraction of the expected solution time.

The relative performance of the two methods is much as expected, each minimizing its objective very effectively. As might have been foreseen the F-W method is more effective in reducing σ^2 than the C-R method is in reducing d , but the reduction of σ^2 with the latter is so impressive that the argument for its use is strongly reinforced.

8. Effects on the LP solution

In the final analysis the argument as to what constitutes good LP scaling can only be resolved by running real-world problems scaled in the various ways and observing their behavior. The basic grounds given for scaling in Section 2 were reduction of the number of simplex iterations required to solve the problem and improved numerical behavior in the simplex algorithm. A small scale test running problems A and B as originally presented and after application of the C-R and F-W algorithms provided no insight whatever—all the runs for each problem were within the normal 5 to 10% variation to be expected under any circumstances. An extension of these trials to the larger problems was ruled out as too expensive. Other practitioners however have certainly observed drastic changes in the number of iterations for some scaling methods, particularly on poorly formulated models.

It is rather difficult to measure improvement in the numerical performance of the simplex algorithm since breakdown of the method is so rare in a properly designed system. One crucial routine in any product form simplex code, however, is the inversion routine, which must be called quite frequently [11]. Since the purpose of re-inversion of the basis is to check the growth of rounding error as well as to minimize the number of non-zero elements in the current representation of the inverse, experiments on the accuracy of this process provide some insight on the numerical effect of scaling. After all, if the original inverse representation is of poor accuracy, the following iterations can hardly be satisfactory.

The inversion scheme used in the UMPIRE system has already been described by the author elsewhere [16]. It can be briefly described as a Gaussian elimination scheme (see Section 3) where the columns are ordered

in such a way as to preserve sparsity, including exploitation of natural triangularity in the basis, and the pivot in each column is chosen to have the smallest row count among the eligible elements within some multiple u of the largest available pivot. The experiment carried out in [16] indicated that a satisfactory value of u is 0.01 (to be increased to 0.1 if significant errors are detected) and that this procedure gives both a very sparse and acceptably accurate inverse representation.

The experiments reported here were carried out as follows: A normal simplex run was carried out for each problem, starting from an advanced basis and saving the basis at the arbitrarily chosen interval of 200 iterations. A total of four bases were saved for each problem, with the exception of problem F for which only two were saved. The problems were then scaled by the Fulkerson–Wolfe and Curtis–Reid algorithms and the saved bases entered in sequence and inverted. We thus have statistics for each problem for the same sequence of bases on a genuine simplex path when the problem is unscaled, and when it has been scaled by the F–W and C–R algorithms.

The statistics from these experiments are very voluminous and are presented here only in summary. Table 8.1 presents the following data for each problem:

M = average number of nonzeros in the bases,

V_a = average number of vectors which can pivot above the bump (see [16]),

V_b = average number of vectors which can pivot below the bump,

S = average number of unit vectors (slacks and artificials) in the bases.

Table 8.1
Basis descriptions—averages

Problem	M	V_a	V_b	S
A	2683	168	260	219
B	4413	165	266	254
C	4869	173	340	239
D	4118	497	437	312
E	13279	678	551	495
F	19424	892	668	661

When inverting the bases we are interested in the growth factor g (see Section 3) and estimates of the relative error in the current solution as well as the number of nonzero elements in the inverse representation. It would

seem reasonable to expect this latter number to be smaller for the scaled bases, since by forcing the elements to be of as nearly comparable size as possible there should be a wider choice of pivots in the eligible size range from which to choose on the grounds of preserving sparsity. The following statistics for each problem are presented in Table 8.2:

- N = the average number of non-pivot non-zeros in the inverse representations,
 p = the average number of pivots,
 $\log \|B\|$ = the average logarithm of the l_∞ norm of the bases,
 $\log \|b\|$ = the average log of the right hand side norms,
 $\log \|r\|$ = the average log of the residual norms,
 $\log b$ = the average log of the growth factor g ,
 $\log e$ = the average log of the estimated relative error in the computed solution $x^{(1)} = B^{-1}b$,
 $\log \kappa(B)$ = the average log of the estimated lower bound on the condition number $\kappa(B)$.

Table 8.2
Inversion statistics—averages

Prob	Scaling	N	p	$\log \ B\ $	$\log \ b\ $	$\log \ r\ $	$\log g$	$\log e$	$\log \kappa(B)$
A	—	2629	554	3.6	3.8	-14.2	3.8	-18.2	3.5
	F-W	2637	550	3.3	4.6	-13.7	3.7	-18.3	3.5
	C-R	2633	550	3.3	2.8	-15.1	3.2	-19.5	3.6
B	—	5301	970	2.7	3.3	-12.3	3.9	-15.5	3.4
	F-W	5536	970	2.0	3.5	-11.5	4.0	-14.4	3.8
	C-R	5585	970	1.8	3.0	-11.4	4.2	-14.8	2.1
C	—	4493	1483	2.8	3.7	-12.4	1.8	-16.3	2.9
	F-W	4491	1483	1.9	2.0	-13.2	2.5	-17.0	3.3
	C-R	4492	1483	1.2	3.0	-13.0	2.0	-17.4	2.6
D	—	3889	1397	4.3	4.0	-11.7	3.2	-15.6	5.3
	F-W	4020	1398	2.5	3.8	-12.5	3.7	-16.4	4.1
	C-R	4051	1397	1.7	3.7	-12.6	3.5	-16.4	2.6
E	—	23025	2763	4.2	5.0	-6.1	4.2	-9.2	6.7
	F-W	24207	2766	3.0	5.2	-6.3	4.6	-8.3	6.5
	C-R	25281	2765	4.4	4.2	-6.7	4.5	-11.0	7.8
F	—	44692	3916	3.6	3.2	-4.7	4.6	-7.5	5.5
	F-W	47509	3915	3.1	5.0	-4.6	5.8	-7.5	6.2
	C-R	49507	3917	2.3	3.6	-6.8	4.7	-8.5	4.4

The last two quantities are estimated by carrying out a step of iterative refinement after each inversion, as in [16], to find

$$\mathbf{r} = \mathbf{b} - B\mathbf{x}^{(1)}, \quad (8.1)$$

$$\delta\mathbf{x} = B^{-1}\mathbf{r}, \quad (8.2)$$

$$\mathbf{x}^{(2)} = \mathbf{x}^{(1)} + \delta\mathbf{x}, \quad (8.3)$$

and

$$e = \frac{\|\delta\mathbf{x}\|}{\|\mathbf{x}^{(2)}\|}. \quad (8.4)$$

A lower bound on $\|B^{-1}\|$ is computed from (8.2) to give a lower bound on $\kappa(B)$ for each basis.

All logarithms are to base 10 and give a convenient estimate of the orders of the numbers involved.

In all the cases considered a relative pivot tolerance of $u = 0.01$ was used. The experiments were in fact repeated for the same range of values of u as in earlier experiments, to see if scaling altered the appropriate value. However this turned out not to be the case in general and the standard setting is used throughout.

Two results not evident from Table 8.2 are of some interest. Two cases were encountered where we were lucky (or unlucky) enough to encounter bases requiring the increase of u from 0.01 to 0.1 to meet the standard system criterion for accuracy (see [16]). The first case involved one of the bases from problem E which was unsatisfactorily inverted for the unscaled problem and greatly improved when the problem was scaled either by the F-W or C-R method. The second case involves a basis for problem F which was satisfactory for the unscaled problem, rather better when C-R scaled, but unsatisfactory when F-W scaled; which is rather unexpected.

9. Discussion

The results in Table 8.2 are surprisingly negative. The increase in accuracy obtained is only of the order of one decimal digit—a not very impressive improvement. Furthermore the growth factors remain almost unchanged by scaling while the estimated lower bounds on the condition numbers are also relatively little affected. However on all these counts we

can conclude that the results for the Curtis–Reid scaling are slightly superior.

An initial surprise is that the number of non-zeros in the inverse representation (N) has not been decreased by scaling, as we might have expected, but *increased*. A possible key to this phenomenon is in the structure of real LP matrices. Kalan [10] has observed that the number of distinct coefficient values in an LP matrix is quite small relative to the total number of coefficients. We are therefore entitled to expect some degree of cancellation in the inversion process, particularly if a large number of $+1$ and -1 coefficients are present. Scaling will, however, destroy this element distribution, and appears to bias the pivot choice in such a way as to reduce cancellation.

The loss of the distribution of unique values due to scaling is a serious difficulty in its own right for those LP systems which use this property to minimize core requirements [10]. However Dickson [5] has recently announced the existence of a scaling algorithm which helps preserve this property. It appears that the use of scaling factors which are themselves powers of a small power of 2 (say 4, 8, or 16) does not seriously increase the number of unique values.

Since the benefits of scaling for our sample problems A–F seem to be highly dubious, and since the arguments used to justify it seem eminently plausible, we must try and explain why this is the case. The answer seems to be in that rather neglected field of mathematical programming—formulation. All of the models in this sample were produced by experienced formulators, who can hopefully be relied on to use sensible units and avoid unnecessarily large or small matrix entries. Thus we see that although the extremes in element magnitudes and their variance can be significantly reduced by scaling, these quantities were not unacceptably large to begin with, and a properly designed LP system can deal with most anomalies through its internal checks and recovery procedures. It is worth noting in passing that given a “friendly” matrix, scaling may in fact distort the problem by seizing on small elements of little real significance in the model and trying to force them up to larger values.

The formulation of these models probably also accounts for the failure of scaling to significantly reduce the number of iterations (though use of Harris’s dynamic scaling [8] has a considerable effect on many of them). As an example of what can happen with less scrupulously formulated models D.C. Rarick kindly supplied the author with the results in Table 9.1 which exhibit the number of iterations required for a model with a few hundred rows using various combinations of row and column scales. (In

Table 9.1
Variation in number of iterations with scaling

Row scaling	Column scaling	Number of iterations
—	—	652
E	E	281
E	E ~ F	332
E	GM	> 800
E	GM ~ F	> 700
GM	E	318
GM	E ~ F	586
GM	GM	580
GM	GM ~ F	436

this table the notation ~F means that free row entries are not used in determining the scaling factors.) The unstable nature of the model is most disconcerting. There are numerous other (though less alarming) examples of problems where scaling has made a significant difference in the number of iterations, but there is often little information about the formulation of the model.

10. Conclusions

In the face of the variation in behavior observed in the previous sections we can only say that if the criteria for good scaling discussed in Section 3 are in fact applicable to solving general systems of linear equations, sparse or otherwise, they do not seem to be completely reliable as a guide for LP problems. It seems that far more fundamental than some mechanized scaling scheme is good formulation. If we happen to know that a model is well formulated, scaling is probably superfluous and in some cases may be detrimental. In the great majority of cases, however, we have little information on the source of the model and the competence of the analyst and some scaling scheme must be available to "iron out" ill-posed problems. From the experimental results to hand the simple geometric mean scaling, optionally followed by equilibration, or the Curtis-Reid method seem to be the most successful.

Acknowledgment

The author is indebted to Laurel Slate for supplying the numerical results summarized in Section 5 and to D.C. Rarick for the results of Table 9.1. The experiments described in Sections 7 and 8 were carried out at Scientific Control Systems, Ltd. The author thanks the company for their support. He is also indebted to E.M.L. Beale, J.J.H. Forrest and J.K. Reid for many helpful discussions.

References

- [1] F.L. Bauer, "Optimally scaled matrices", *Numerische Mathematik* 5 (1963) 73–87.
- [2] E.M.L. Beale, *Mathematical programming in practice* (Pitman, London, 1968).
- [3] J. de Buchet, "Expériences et statistiques sur la résolution des programmes linéaires de grandes dimensions", in: D.B. Hertz and J. Melese, eds., *Proceedings 4th IFORS conference* (Wiley, New York, 1966) pp. 3–12.
- [4] A.R. Curtis and J.K. Reid, "On the automatic scaling of matrices for Gaussian elimination", *Journal of the Institute of Mathematics and its Applications* 10 (1972) 118–124.
- [5] J.C. Dickson, "On keeping both storage and I/O requirements low in linear programming", paper presented to the VIII International Symposium on Mathematical Programming (1973) (abstract).
- [6] D.R. Fulkerson and P. Wolfe, "An algorithm for scaling matrices", *SIAM Review* 4 (1962) 142–146.
- [7] R.W. Hamming, *Introduction to applied numerical analysis* (McGraw-Hill, New York, 1971).
- [8] P.M.J. Harris, "Pivot selection methods of the Devex LP code", *Mathematical Programming Study* 4 (1974) 30–57 (this volume).
- [9] G. Hentges, "MPS/360-matrix scaling", IBM France research memorandum (March 1969).
- [10] J.E. Kalan, "Aspects of large-scale in-core linear programming", in: *ACM 1971 annual conference proceedings*.
- [11] W. Orchard-Hays, *Advanced linear programming computing techniques* (McGraw-Hill, New York, 1968).
- [12] J.K. Reid, "A note on the stability of Gaussian elimination", *Journal of the Institute of Mathematics and its Applications* 8 (1971) 374–375.
- [13] L. Slate, Private communication (November 1972).
- [14] A. van der Sluis, "Condition numbers and equilibration of matrices", *Numerische Mathematik* 14 (1969) 14–23.
- [15] A. van der Sluis, "Condition, equilibration and pivoting in linear algebraic systems", *Numerische Mathematik* 15 (1970) 74–86.
- [16] J.A. Tomlin, "Pivoting for size and sparsity in linear programming inversion routines", *Journal of the Institute of Mathematics and its Applications* 10 (1972) 289–295.
- [17] J.H. Wilkinson, *The algebraic eigenvalue problem* (Oxford University Press, Oxford, 1965).